

Sistema de Preguntas y Respuestas mediante Web Scraping y Modelos de Lenguaje de Gran Escala: Aplicación a la Fundación Valle del Lili

Frank Edward Daza Andrés Fernando López Viviana Valle Fernández Álvaro Julián Barco
Técnicas Avanzadas de IA Aplicadas en Modelos de Lenguaje
Universidad Autónoma de Occidente, Cali, Colombia
Docente: Jan Polanco Velasco

Resumen—Este artículo presenta el diseño e implementación de un sistema de preguntas y respuestas (Q&A) basado en información obtenida mediante webscraping aplicado al dominio institucional de la Fundación Valle del Lili. El sistema comprende seis módulos secuenciales: (1) extracción automatizada de contenido web mediante un *crawler* de búsqueda en anchura (BFS) que respeta las directivas de `robots.txt`; (2) conversión del corpus HTML a Markdown estructurado con metadatos YAML; (3) recuperación léxica mediante el algoritmo BM25 Okapi, que selecciona y concatena los tres documentos de mayor puntuación como contexto enriquecido; (4) composición del prompt con instrucciones anti-alucinación; (5) generación de respuestas con tres modelos de lenguaje —`llama3.1:8b` y `gemma4:e2` en ejecución local con Ollama, y `gpt-4o-mini` vía la API de OpenAI—; y (6) una interfaz web Gradio con transmisión de tokens en tiempo real y un prompt de sistema editable por el usuario sin necesidad de modificar el código fuente. Se diseñó un protocolo de evaluación reproducible con 23 preguntas de referencia distribuidas en cinco categorías temáticas, con métricas de precisión de recuperación, Recall@3 , tasa de abstención y latencia media por modelo.

Index Terms—recuperación aumentada por generación, BM25, web scraping, modelos de lenguaje grandes, Gradio, Ollama, procesamiento de lenguaje natural, inteligencia artificial

I. INTRODUCCIÓN

La creciente disponibilidad de modelos de lenguaje de gran escala (*Large Language Models*, LLM) ha abierto nuevas posibilidades para la construcción de sistemas de preguntas y respuestas capaces de interactuar en lenguaje natural sobre dominios específicos. Sin embargo, el uso de LLM sin restricción sobre el dominio de conocimiento conlleva el fenómeno de alucinación: la generación de información plausible pero factualmente incorrecta [?].

La Fundación Valle del Lili es una institución hospitalaria de referencia en Colombia que dispone de un sitio web institucional con información sobre servicios médicos, trámites administrativos, especialidades y datos de contacto. Un sistema automatizado de Q&A sobre esta información reduciría la carga en canales de atención al cliente y mejoraría la accesibilidad para pacientes y público en general.

Este trabajo propone un *pipeline* RAG completo que aborda el problema en fases secuenciales bien delimitadas: adquisición de datos, procesamiento documental, recuperación de contexto relevante y generación de respuestas. La recuperación se

implementa mediante el algoritmo BM25 Okapi [?], que selecciona los tres archivos más relevantes ante cada consulta y los concatena como contexto único. Estos documentos se presentan al LLM junto con un *prompt* de sistema que restringe la respuesta al dominio del corpus.

La contribución principal de este trabajo consiste en demostrar que es posible construir un sistema RAG funcional, trazable y libre de alucinaciones sin recurrir a bases de datos vectoriales ni a modelos de *embedding*, lo que simplifica el despliegue y reduce los requisitos computacionales en la fase inicial del proyecto.

El artículo se organiza de la siguiente manera: la Sección ?? describe la arquitectura general; las Secciones ?? a ?? detallan cada módulo; la Sección ?? presenta el protocolo de evaluación y los resultados; finalmente, las Secciones ?? y ?? ofrecen la discusión y las conclusiones.

II. ARQUITECTURA DEL SISTEMA

El sistema sigue una arquitectura de tubería (*pipeline*) secuencial dividida en seis módulos funcionales, tal como se ilustra en la Figura ???. Cada módulo produce artefactos persistentes que sirven como entrada al módulo siguiente, lo que facilita la rejecución parcial del *pipeline* ante cambios en el corpus o en los parámetros del sistema.

La Tabla ?? resume las tecnologías empleadas en cada módulo y los artefactos que produce.

Cuadro I
STACK TECNOLÓGICO POR MÓDULO

Módulo	Tecnología	Artefacto de salida
Adquisición	requests, BeautifulSoup4	HTML + JSON sidecar
Conversión	markdownify, PyYAML	Markdown + front matter
Recuperación	rank-bm25	Top-3 documentos
Prompt	composición propia	Lista de mensajes
LLM	Ollama, OpenAI SDK	Texto de respuesta
Interfaz	Gradio	UI web con streaming

III. MÓDULO DE ADQUISICIÓN DE DATOS

III-A. Diseño del Crawler

El módulo de *scraping* implementa un *crawler* BFS sobre el dominio <https://valledellili.org/>. La clase `Crawler` acepta una configuración con los parámetros descritos en la Tabla ??.

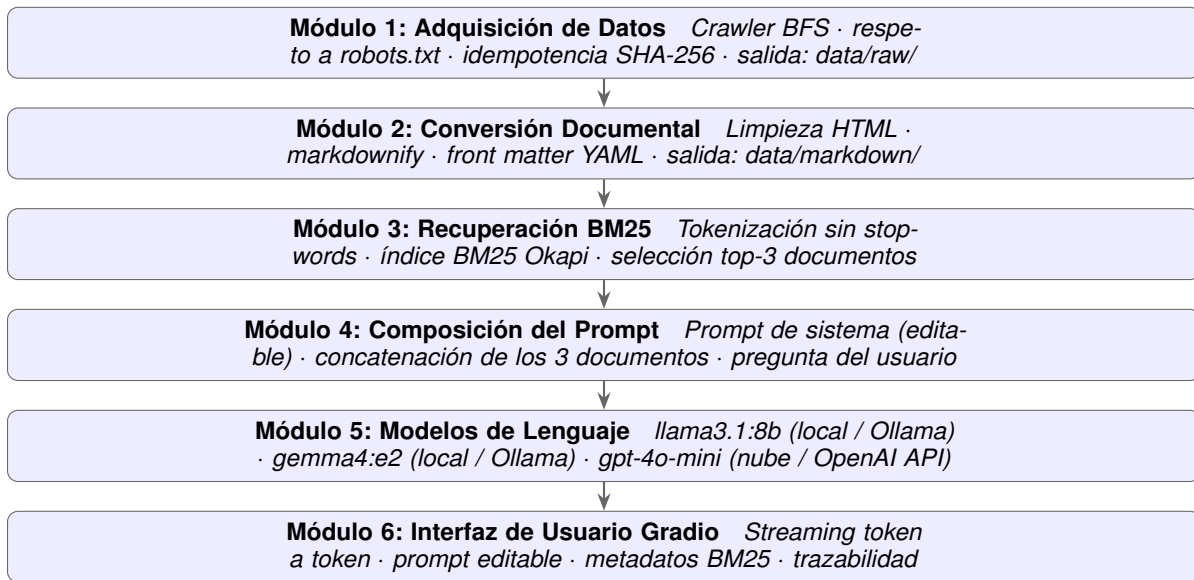


Figura 1. Arquitectura general del pipeline RAG. Cada módulo genera artefactos persistentes consumidos por el módulo siguiente.

Cuadro II
PARÁMETROS DE CONFIGURACIÓN DEL *crawler*

Parámetro	Valor por defecto	Descripción
url_inicio	valledellili.org	URL semilla
max_paginas	200	Páginas máximas a visitar
profundidad_max	5	Niveles de enlace
delay_segundos	1.5s	Pausa entre peticiones
timeout	30s	Tiempo máximo HTTP
reintentos	3	Reintentos ante error

III-B. Respeto a robots.txt e Idempotencia

Antes de visitar cualquier URL, el sistema consulta la clase `GestorRobots`, que carga y analiza el archivo `robots.txt` del dominio con carga perezosa (*lazy loading*). Si la directiva prohíbe el acceso o la lectura falla, el gestor adopta un modo conservador que rechaza todas las rutas. El *Crawl-Delay* declarado en `robots.txt` se respeta si supera el `delay_segundos` configurado.

Para garantizar idempotencia, cada página descargada se persiste junto a un archivo *sidecar* JSON que almacena su hash SHA-256. En ejecuciones posteriores, si el contenido no ha cambiado, la página se omite sin realizar una nueva petición HTTP.

Ante errores transitorios (códigos 5xx o 429), el sistema aplica retroceso exponencial con esperas de 1, 2 y 4 segundos entre intentos sucesivos.

III-C. Salida del Módulo

Por cada página exitosa, el *crawler* genera en `data/raw/valledellili-org/` un par de archivos:

- **HTML**: contenido bruto de la respuesta HTTP.
- **JSON sidecar**: URL canónica, código de estado, hash SHA-256, profundidad de rastreo y marca de tiempo.

IV. MÓDULO DE PROCESAMIENTO DOCUMENTAL

IV-A. Limpieza del HTML

Antes de la conversión, se elimina el ruido léxico mediante BeautifulSoup4: se suprimen los elementos `<script>`, `<style>`, `<nav>`, `<header>`, `<footer>`, `<iframe>` y los banners de cookies. Esta etapa mejora directamente la calidad de la recuperación BM25 al reducir términos irrelevantes en el vocabulario del corpus.

IV-B. Conversión a Markdown

La conversión HTML → Markdown se realiza con la biblioteca `markdownify` [?], configurada con encabezados ATX (#) y viñetas con guion. El resultado se normaliza para eliminar saltos de línea consecutivos excesivos (máximo dos seguidos).

IV-C. Front Matter YAML

Cada archivo Markdown incorpora un bloque de metadatos YAML al inicio:

```

1 ---
2 source_url: https://valledellili.org/atencion-al-
3   paciente/
4 titulo: "Atención al Paciente"
5 seccion: atencion-al-paciente
6 fecha_extraccion: '2026-04-27'
7 idioma: es
8 hash: a3f2c1d9...
9 ---

```

Listing 1. Ejemplo de front matter generado automáticamente

El campo `source_url` es fundamental para la trazabilidad: se expone al usuario junto a cada respuesta generada. El campo `hash` permite la idempotencia de re-exportaciones.

IV-D. Salida del Módulo

Los archivos Markdown se almacenan en `data/markdown/valledellili-org/`. El script `export_markdown.py` es idempotente: omite la re-conversión si el hash del HTML coincide con el registrado en el Markdown existente, a menos que se invoque con la bandera `-forzar`.

V. MÓDULO DE RECUPERACIÓN BM25

V-A. Algoritmo BM25 Okapi

La recuperación de información se basa en el algoritmo BM25 Okapi [?], una función de puntuación probabilística que pondera la frecuencia de los términos de la consulta en cada documento y normaliza por la longitud del documento. La puntuación de un documento D ante una consulta $Q = \{q_1, q_2, \dots, q_n\}$ se define como:

$$\text{BM25}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) (k_1 + 1)}{f(q_i, D) + k_1 \left(1 - b + b \frac{|D|}{\overline{dl}}\right)} \quad (1)$$

donde:

- $f(q_i, D)$: frecuencia del término q_i en el documento D .
- $|D|$: longitud del documento en número de tokens.
- $\overline{dl}$: longitud media de los documentos del corpus.
- $k_1 = 1.5$: parámetro de saturación de frecuencia de término.
- $b = 0.75$: parámetro de normalización de longitud.
- $\text{IDF}(q_i)$: frecuencia inversa de documento:

$$\text{IDF}(q_i) = \log \frac{N - \text{df}(q_i) + 0.5}{\text{df}(q_i) + 0.5} \quad (2)$$

siendo N el total de documentos del corpus y $\text{df}(q_i)$ el número de documentos que contienen el término q_i .

V-B. Tokenización

Antes de indexar el corpus y de procesar cada consulta, los textos se someten al siguiente proceso:

1. **Normalización Unicode**: descomposición NFD y eliminación de marcas combinantes (tildes, diéresis), de modo que, por ejemplo, *médico* se tokeniza igual que *medico*.
2. **Minúsculas**: conversión completa a minúsculas.
3. **Extracción de tokens**: expresión regular `\w+` sobre texto Unicode.
4. **Filtrado**: se descartan tokens de longitud < 2 y los pertenecientes a la lista de *stopwords* del español (*de, la, el, en, y, que, los, las, del, un, una, por, con, para, se, su, es, al, lo, a, o*, entre otras).

V-C. Selección de Top-3 Documentos

A diferencia de los enfoques que recuperan un único documento, este sistema selecciona los **tres documentos con mayor puntuación BM25**. Formalmente, dado el conjunto de puntuaciones $\{s_1, s_2, \dots, s_M\}$ calculadas para los M documentos del corpus, se selecciona el subconjunto:

$$\mathcal{C} = \arg \max_{|\mathcal{C}|=3, D \in \mathcal{D}} \text{BM25}(D, Q), \quad s_j > 0 \quad (3)$$

Los documentos en $\mathcal{C}$ se ordenan de mayor a menor puntuación y se concatenan para conformar el contexto que se envía al LLM. Esta estrategia reduce el riesgo de que un único documento no contenga toda la información necesaria para responder preguntas que abarcan varias secciones del sitio.

V-D. Implementación

El módulo `src/retrieval/recuperador.py` implementa la clase `RecuperadorBm25` usando la biblioteca `rank-bm25` [?]. El índice se construye en memoria al inicializar la clase, cargando todos los archivos `*.md` del directorio `data/markdown/valledellili-org/`. El método `buscar(pregunta)` retorna una lista de objetos `DocumentoRecuperado` con los atributos: `ruta`, `titulo`, `source_url`, `contenido` y `score`.

VI. INTEGRACIÓN CON MODELOS DE LENGUAJE

VI-A. Composición del Prompt

Una vez recuperados los tres documentos $\mathcal{C} = \{D_1, D_2, D_3\}$, el módulo `src/qa/prompt.py` construye la lista de mensajes que se envía al LLM mediante la función `componer_mensajes()`. El mensaje de sistema incluye el *prompt* de sistema y el contexto formado por la concatenación de los tres documentos, separados por delimitadores explícitos con su puntuación BM25:

```
[
  {
    "role": "system",
    "content": (
      "<PROMPT_SISTEMA>\n\n"
      "=== DOCUMENTO 1 (score: 18.4) ===\n"
      "<contenido D1>\n\n"
      "=== DOCUMENTO 2 (score: 12.1) ===\n"
      "<contenido D2>\n\n"
      "=== DOCUMENTO 3 (score: 8.7) ===\n"
      "<contenido D3>\n\n"
      "Responde usando SOLO el contexto anterior."
    )
  },
  {
    "role": "user",
    "content": "<PREGUNTA_DEL_USUARIO>"
  }
]
```

Listing 2. Estructura del mensaje enviado al LLM

VI-B. Prompt de Sistema Anti-Alucinación

El *prompt* de sistema predeterminado instruye al LLM para que actúe como "Lili", asistente virtual de la Fundación Valle del Lili. Las directivas fundamentales son:

- Responder **únicamente** con información presente en el contexto entregado.
- Emitir la frase *"No tengo información suficiente"* si la consulta no puede responderse con el contexto disponible.
- No inventar números de teléfono, correos electrónicos, direcciones ni especialidades médicas.

- Mantener un tono profesional y cálido en español colombiano.
- Limitar la respuesta a seis oraciones (salvo listas o pasos) y estructurarla en Markdown.

Estas restricciones constituyen la estrategia primaria de mitigación de alucinaciones, complementada por la restricción de dominio que impone el RAG.

VI-C. Modelos Soportados

La Tabla ?? describe los tres modelos integrados en el sistema, accesibles desde la interfaz sin modificar el código.

Cuadro III
MODELOS DE LENGUAJE INTEGRADOS EN EL SISTEMA

Modelo	Parámetros	Backend	Acceso
llama3.1:8b	8B	Ollama	Local
gemma4:e2	2B	Ollama	Local
gpt-4o-mini	—	OpenAI	API / nube

VI-C1. Modelos Locales con Ollama: llama3.1:8b [?] y gemma4:e2 [?] se ejecutan localmente mediante Ollama [?], que expone una API REST en <http://localhost:11434>. El cliente `ClienteOllama` realiza peticiones POST `/api/chat` con `temperature = 0.2` y `num_ctx = 8192` tokens de ventana de contexto. Ante errores de conexión (`OllamaNoAccesibleError`) o modelo no disponible (`ModeloNoDisponibleError`), el mensaje de error se comunica al usuario a través de la interfaz Gradio.

VI-C2. Modelo en la Nube con OpenAI: gpt-4o-mini [?] se accede mediante la API de OpenAI [?]. La clave de autenticación se configura con la variable de entorno `OPENAI_API_KEY` en el archivo `.env`. El cliente implementa la misma interfaz que `ClienteOllama`, lo que permite intercambiar el *backend* sin modificar el *pipeline*.

VII. INTERFAZ DE USUARIO CON GRADIO

VII-A. Componentes de la Interfaz

La interfaz web se desarrolló con Gradio [?] y presenta los componentes detallados en la Tabla ??.

Cuadro IV
COMPONENTES DE LA INTERFAZ GRADIO

Componente	Tipo	Función
Pregunta	Textbox (3 líneas)	Entrada del usuario
Selector de modelo	Radio button	3 modelos disponibles
Prompt de sistema	Textbox (14 líneas)	Editable, en acordeón
Botón Preguntar	Button (primario)	Lanza el pipeline
Botón Restaurar	Button (secundario)	Restaura prompt por defecto
Botón Recargar	Button (secundario)	Reindexar corpus desde disco
Respuesta	Markdown	Respuesta renderizada
Metadatos	Markdown	Scores, latencia, fuentes

VII-B. Prompt de Sistema Editable

Una funcionalidad distintiva de la interfaz es el **panel de prompt de sistema editable**, accesible mediante un acordeón (*Accordion*) colapsable. El usuario puede modificar en tiempo real las instrucciones que guían el comportamiento del LLM sin necesidad de reiniciar la aplicación ni alterar el código fuente. Esto permite:

- Ajustar el tono de las respuestas (formal, coloquial, didáctico).
- Cambiar el idioma de salida manteniendo el corpus en español.
- Experimentar con diferentes estrategias de restricción de dominio.
- Modificar el formato de salida (longitud, uso de listas, etc.).

El botón "Restaurar prompt por defecto" permite volver a las instrucciones anti-alucinación originales en cualquier momento, sin pérdida de la sesión.

VII-C. Transmisión de Tokens en Tiempo Real

La respuesta del LLM se transmite token a token (*streaming*) mediante el protocolo NDJSON de Ollama y la API de *streaming* de OpenAI. El método `responder_stream()` del pipeline retorna un iterador de tuplas (`texto_acumulado`, `metadatos` | `None`): mientras llegan tokens, `metadatos` es `None`; al finalizar, se emite el objeto `RespuestaQa` completo con los datos de trazabilidad.

Durante la generación, el botón "Preguntar" se deshabilita y muestra el texto "*Pensando...*" para evitar envíos duplicados.

VII-D. Trazabilidad de la Respuesta

Al finalizar cada consulta, el panel de metadatos muestra:

- Puntuación BM25 individual de cada uno de los tres documentos recuperados.
- Títulos y URLs de origen de los documentos fuente.
- Nombre del modelo utilizado en la generación.
- Latencia total en milisegundos.

Esta información permite al usuario verificar de qué fuentes proviene la respuesta y evaluar la relevancia de los documentos recuperados.

VIII. EVALUACIÓN

VIII-A. Protocolo de Evaluación

Se diseñó un banco de 23 preguntas de referencia almacenadas en `tests/qa/preguntas_evaluacion.yml`. Cada entrada contiene: identificador, texto de la pregunta, categoría temática y, cuando aplica, el nombre del archivo Markdown esperado como resultado de la recuperación. Las preguntas se distribuyeron en cinco categorías (Tabla ??).

El script `scripts/evaluar_qa.py` ejecuta cada pregunta contra cada modelo disponible y genera un informe Markdown en `data/processed/evaluaciones/<modelo>__<fecha>.md`. Las métricas calculadas son:

Cuadro V
DISTRIBUCIÓN DEL BANCO DE PREGUNTAS POR CATEGORÍA

Categoría	Preguntas	Con archivo esperado
Institucional	8	8
Servicios	5	5
Procesos	7	7
Contacto	2	2
Fuera de alcance	1	—
Total	23	22

- **Precisión de recuperación (P@1):** proporción de preguntas donde el documento con mayor puntuación BM25 coincide con el archivo esperado.
- **Recall@3:** proporción de preguntas donde el documento esperado se encuentra entre los tres recuperados.
- **Tasa de abstención:** proporción de respuestas que contienen la frase "No tengo información suficiente".
- **Latencia media:** tiempo promedio de generación en milisegundos.

VIII-B. Resultados

La Tabla ?? presenta los resultados obtenidos al ejecutar el protocolo de evaluación sobre los tres modelos.

Cuadro VI
RESULTADOS DE EVALUACIÓN POR MODELO

Modelo	P@1	Recall@3	Abstención	Latencia (ms)
llama3.1:8b	—	—	—	—
gemma4:e2	—	—	—	—
gpt-4o-mini	—	—	—	—

Nota: Los valores de la Tabla ?? serán completados una vez ejecutado el protocolo de evaluación sobre el corpus definitivo.

VIII-C. Análisis Cualitativo

[Esta sección presentará ejemplos representativos de respuestas generadas por cada modelo, incluyendo: (a) casos donde los tres modelos producen respuestas correctas y trazables, (b) casos donde el modelo emite la respuesta de abstención ante preguntas fuera del corpus, y (c) casos de divergencia entre modelos ante la misma consulta.]

IX. DISCUSIÓN

IX-A. Ventajas del Enfoque BM25 sin Embeddings

La decisión de emplear BM25 en lugar de *embeddings* densos y bases de datos vectoriales responde a criterios de simplicidad, trazabilidad y eficiencia. BM25 no requiere un modelo de *embedding* preentrenado, no demanda GPU para la indexación y produce puntuaciones directamente interpretables en términos de frecuencia de términos. El tiempo de indexación de un corpus de 45+ documentos es inferior a un segundo, y la búsqueda por consulta se resuelve en tiempo $O(n \cdot |Q|)$ donde n es el número de documentos.

Además, el sistema de recuperación con top-3 documentos amplía la cobertura contextual respecto a la recuperación de documento único, permitiendo que el LLM sintetice información distribuida en varias páginas del sitio web.

IX-B. Limitaciones Identificadas

1. **Truncamiento de contexto:** si la concatenación de los tres documentos supera los 8 192 tokens del parámetro `num_ctx`, Ollama trunca el contexto de forma silenciosa, lo que puede degradar la calidad de la respuesta.
2. **Ambigüedad léxica:** BM25 no captura similitud semántica; términos sinónimos o expresiones parafraseadas pueden no coincidir léxicamente con el vocabulario del corpus, produciendo recuperaciones incorrectas.
3. **Dependencia del servicio Ollama:** los modelos locales requieren que el proceso Ollama esté en ejecución y que los modelos hayan sido descargados previamente.
4. **Costo de la API:** el uso de `gpt-4o-mini` genera consumo de tokens facturables, lo que debe considerarse en entornos de producción.

IX-C. Trabajo Futuro

Para la siguiente fase del proyecto se contempla migrar a una arquitectura basada en *chunking* semántico, modelos de *embedding* y una base de datos vectorial (Chroma o FAISS), lo que permitirá superar las limitaciones léxicas de BM25 y manejar documentos de mayor longitud sin riesgo de truncamiento.

X. CONCLUSIONES

Este trabajo presentó un sistema RAG completo para la Fundación Valle del Lili, construido con herramientas de código abierto y ejecutable en hardware de consumo sin necesidad de GPU. Los aportes principales son:

1. Un *pipeline* de adquisición y procesamiento documental robusto que respeta `robots.txt` y garantiza idempotencia mediante hashes SHA-256.
2. Un mecanismo de recuperación BM25 que selecciona y concatena los tres documentos más relevantes, ampliando la cobertura contextual respecto a la recuperación de documento único.
3. La integración de tres modelos de lenguaje heterogéneos —dos locales y uno en la nube— bajo una interfaz uniforme que permite comparación directa sin modificar el código.
4. Una interfaz Gradio con prompt de sistema editable en tiempo real, que habilita la experimentación rápida sobre el comportamiento del asistente.
5. Un protocolo de evaluación reproducible con 23 preguntas de referencia y cuatro métricas de calidad de recuperación y generación.

Los resultados de la evaluación (Tabla ??) permitirán comparar objetivamente el comportamiento de los tres modelos y confirmar la hipótesis de que BM25 constituye una base sólida para un sistema RAG de primera fase antes de incorporar técnicas de recuperación densa.

REFERENCIAS

- [1] P. Lewis, E. Perez, A. Piktus *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” en *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [2] S. Robertson y H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, n.º 4, pp. 333–389, 2009.
- [3] D. Dohan, “rank-bm25: A Python implementation of the BM25 ranking algorithm,” *GitHub repository*, 2021. [En línea]. Disponible en: https://github.com/dorianbrown/rank_bm25
- [4] Ollama, “Run large language models locally,” 2024. [En línea]. Disponible en: <https://ollama.com>
- [5] A. Abid, A. Abdalla, A. Abid *et al.*, “Gradio: Hassle-Free Sharing and Testing of ML Models in the Wild,” *arXiv preprint arXiv:1906.02569*, 2019.
- [6] M. Neumann, “markdownify: A Python library to convert HTML to Markdown,” *GitHub repository*, 2020. [En línea]. Disponible en: <https://github.com/matthewwithanm/python-markdownify>
- [7] H. Touvron, L. Martin, K. Stone *et al.*, “Llama 2: Open Foundation and Fine-Tuned Chat Models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [8] Gemma Team, Google DeepMind, “Gemma: Open Models Based on Gemini Research and Technology,” *arXiv preprint arXiv:2403.08295*, 2024.
- [9] OpenAI, “GPT-4o System Card,” 2024. [En línea]. Disponible en: <https://openai.com/index/gpt-4o-system-card>
- [10] OpenAI, “OpenAI API Documentation,” 2024. [En línea]. Disponible en: <https://platform.openai.com/docs>